

Thiese update 6/23

research of IP reuse on realbench

Recap

last time we build a multi-level retrieval and caching assistance for RTL coding agent, and test on verilog-Eval & RTLLM.

And to embody the value of this, we want to apply it on a real-world IP reuse scenario now.

Outline

- **Last time Action item: how RAG samples affect generation**
- Feature of a hierarchical hardware design
- Realbench
- agentic IP-reuse RTL generation pipeline **Prototype**
 - Planning
 - Generate & fix (with RAG)
- **Experiment**
 - pure model v.s. prototype
 - issue & fix
 - v2 pipeline
 - repair cache ablaiton
 - error analysis
 - capability test
 - system level test
- **Action Items**

Benchmark test intro

- RAG is a **outer information support system** designed to help the LLM's generation to **avoid from hallucination** and supplement sufficient knowledge.
- Which means for a verilog generation task, if use RAG, then it should supply a similar enough sample as reference or even give the result directly.
- Also for a real world verilog coding spec, the hierarchical module structure more common rather than a simple single module hardware.
- When decomposing a complex module into several submodules, the following situation may occur: several modules may be **built from the same submodule**.
- If we can correctly reuse this module (or IP), we can accelerate the generation process and save lots of resources.

Benchmark test

- For those correct, does RAG get same document?
 - determine that model can pass the test is because of it's own ability or the correct reference
- For those incorrect, what RAG retrieve, how similar is it?
 - determine low quality reference will affect result or not.
- So we check the generation result of siliconMind-v1 with **pure model & rag mode**.
- In case study, we catagorize the error type of each generation fail as following:
 - S = Syntax Error
 - C = Compile Error
 - F = Functional Error
 - G = Generate Error (no code been parsed)
 - R = Runtime Failure

Benchmark test

- Prob028_m2014_q4a

	Pass Rate	Retrieve Similarity
Pure Model	40%, 2S1C	X
RAG Mode	80%, 1C	0.885

I would like you to implement a module named TopModule with the following interface. All input and output ports are one bit unless otherwise specified.

- input d
- input ena
- output q

The module should implement a **D latch** using an always block.

```
`default_nettype none

module top_module (
    input d,
    input ena,
    output reg q
);
    always @(*) begin
        if (ena) q = d;
    end
endmodule
```

Design a level-sensitive **D latch** in Verilog with the following specifications:

****Module name:**** `top\_module`
****Port list (exact order and widths):****

- `input d` : single-bit data input.
- `input ena` : single-bit enable (gate) input. When `ena` is high the latch should be transparent, i.e., the output follows `d`. When `ena` is low the latch must hold its previous value.
- `output reg q` : single-bit latched output.

* The latch must be described using a single combinational `always @(\*)` block. Inside this block, assign `q` only when `ena` is high (use a blocking assignment). Do ****not**** provide an explicit `else` branch; the absence of an assignment when `ena` is low should cause a latch to be inferred that retains its state.

* Include the compiler directive `` `default\_nettype none `` before the module declaration to require all nets to be explicitly declared.

Write the complete Verilog code that satisfies these requirements. The code you produce must be exactly the one shown in the snippet below.

Benchmark test

- [Prob150_review2015_fsmonehot](#)
 - The question is ask for a **combinational part of a one-hot encoded Moore FSM**.
- [Retrieve document](#)
 - The question asks for a **complete FSM controller**, which mixed Moore & Mealy FSM
- And there's also many different detail
 - clk, reset, even I/O is very different.

	Pass Rate	Retrieve Similarity
Pure Model	100%	X
RAG Mode	0%, 3R2G	0.614

I would like you to implement a module named TopModule with the following interface. All input and output ports are one bit unless otherwise specified.

- input d
- input done_counting
- input ack
- input state (10 bits)
- output B3_next
- output S_next
- output S1_next
- output Count_next
- output Wait_next
- output done
- output counting
- output shift_ena

The module should implement the following Moore state machine with 3 input (d, done_counting, ack) and 3 outputs (shift_ena, counting, done). Unless otherwise stated in the diagram below, assume outputs are 0 and inputs are don't cares.

state	(output)	--input--> next state
-------	----------	-----------------------

S	()	--d=0--> S
S	()	--d=1--> S1
S1	()	--d=0--> S
S1	()	--d=1--> S11
S11	()	--d=0--> S110
S11	()	--d=1--> S11
S110	()	--d=0--> S
S110	()	--d=1--> B0
B0	(shift_ena=1)	--(always go to next cycle)--> B1
B1	(shift_ena=1)	--(always go to next cycle)--> B2
B2	(shift_ena=1)	--(always go to next cycle)--> B3
B3	(shift_ena=1)	--(always go to next cycle)--> Count
Count	(counting=1)	--done_counting=0--> Count
Count	(counting=1)	--done_counting=1--> Wait
Wait	(done=1)	--ack=0--> Wait
Wait	(done=1)	--ack=1--> S

At reset, the state machine starts in state "S". Derive next-state logic equations and output logic equations by inspection assuming the following one-hot encoding is used: (S, S1, S11, S110, B0, B1, B2, B3, Count, Wait) = (10'b0000000001, 10'b0000000010, 10'b00000000100, ... , 10'b1000000000)

Derive state transition and output logic equations by inspection assuming a one-hot encoding. Implement only the state transition logic and output logic (the combinational logic portion) for this state machine.

Write code that generates the following signals:

- B3_next -- Assert when next-state is B3 state
- S_next -- Assert when next-state is S state
- S1_next -- Assert when next-state is S1 state
- Count_next -- Assert when next-state is Count state
- Wait_next -- Assert when next-state is Wait state
- done -- output logic
- counting -- output logic
- shift_ena -- output logic

Design the control unit for a **sequential binary multiplier** using the **one-flip-flop-per-state** coding technique.

The controller must manage three distinct phases of the multiplication process:

- Idle (ready)** – The multiplier is waiting for a new operation.
- Load (decr_p)** – The multiplicand and multiplier registers are loaded.
- Shift/Multiply (shift_regs)** – The registers are repeatedly shifted and, when appropriate, the partial product is added.

The required behaviour is:

- The FSM is reset to the idle state when the active-low synchronous reset (``reset``) is asserted.
- While in the idle state (``ready`` = 1), asserting the ``start`` input should move the FSM to the **load** state.
- After one clock cycle in the load state, the FSM should automatically advance to the **shift** state.
- The FSM stays in the shift state until the input ``zero`` becomes high, indicating that the multiplication is finished; at that moment it should return to the idle state on the next clock edge.
- Moore outputs** (``ready``, ``decr_p``, ``shift_regs``) must reflect the present state only.
- Mealy outputs**:
 - ``load_regs`` must be high when the controller is in the idle state **and** ``start`` is high (i.e., the moment the multiplication begins).
 - ``add_regs`` must be high when the controller is in the load state **and** the least-significant bit of the multiplier (``Q0``) is high (i.e., a partial-product addition should occur).

Implement the controller as a Verilog module named **control_multiplier_2** with the following port list:

Signal	Direction	Width	Description
<code>`start`</code>	input	1	Starts a new multiplication
<code>`zero`</code>	input	1	Indicates that the product has become zero (end of multiplication)
<code>`Q0`</code>	input	1	LSB of the multiplier operand (used for addition)
<code>`clk`</code>	input	1	System clock (positive edge)
<code>`reset`</code>	input	1	Active-low synchronous reset
<code>`ready`</code>	output	1	Moore output – high in idle state
<code>`decr_p`</code>	output	1	Moore output – high in load state
<code>`shift_regs`</code>	output	1	Moore output – high in shift state
<code>`load_regs`</code>	output	1	Mealy output – high when <code>`ready`</code> & <code>`start`</code>
<code>`add_regs`</code>	output	1	Mealy output – high when <code>`decr_p`</code> & <code>`Q0`</code>

Use primitive gate primitives (``and``, ``or``, ``buf``) for the combinational next-state logic and a single ``always @(posedge clk)`` block for the sequential state register. The reset should be handled synchronously inside the clocked block (i.e., ``if (!reset) state <= 3'b000``).

<pre> module control_multiplier_2(input start, input zero, input Q0, input clk, input reset, output ready, output decr_p, output shift_regs, output load_regs, output add_regs); reg [2:0] state; wire ns0, ns1, ns2; wire s0_not_start, s2_zero, s0_start, s2_not_zero; and(s0_not_start, ~start, state[0]); and(s2_zero, zero, state[2]); or (ns0, s0_not_start, s2_zero); and(s0_start, state[0], start); buf(ns1, s0_start); </pre>	<pre> and(s2_not_zero, ~zero, state[2]); or (ns2, state[1], s2_not_zero); always @(posedge clk) begin if (!reset) state <= 3'b001; else begin state[0] <= ns0; state[1] <= ns1; state[2] <= ns2; end end assign ready = state[0]; assign decr_p = state[1]; assign shift_regs = state[2]; and(load_regs, state[0], start); and(add_regs, state[1], Q0); endmodule </pre>
---	---

Benchamrk Test - Conclusion

- So we can see that, a good retrieve sample can help LLM to generate a correct result.
- But also a bad retrieve sample may mislead the model to become confused about what we're going to do.
- Conclusion: Add a **hard similarity threshold** for retrieved sample, only reference these samples when it is good/similar enough to help understanding the question. Else should fallback to generate by LLM itself.

Outline

- Last time Action item: how RAG samples affect generation
- **Feature of a hierarchical hardware design**
- Realbench
- agentic IP-reuse RTL generation pipeline **Prototype**
 - Planning
 - Generate & fix (with RAG)
- **Experiment**
 - pure model v.s. prototype
 - issue & fix
 - v2 pipeline
 - repair cache ablation
 - error analysis
 - capability test
 - system level test
- **Action Items**

IP reuse feature & Solution

1. Spec is usually too large to be process in one-shot (700k characters)
2. Current verilog gen model can hard to directly generate the correct code.

As for solution:

1. Use a large context orchestrator do context catagorize and splits into multiple document for multiple agent parallel code generation.
2. Build a hierarchical graph structure to store the state of decomposition module and trace for code build.

Outline

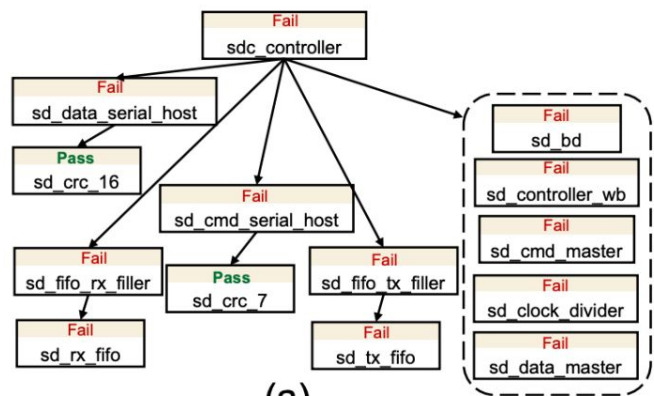
- Last time Action item: how RAG samples affect generation
- Feature of a hierarchical hardware design
- **Realbench**
- agentic IP-reuse RTL generation pipeline **Prototype**
 - Planning
 - Generate & fix (with RAG)
- **Experiment**
 - pure model v.s. prototype
 - issue & fix
 - v2 pipeline
 - repair cache ablation
 - error analysis
 - capability test
 - system level test
- **Action Items**

RealBench^[1]

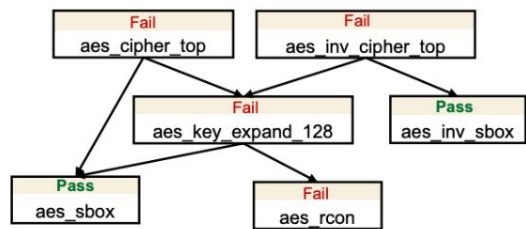
- RealBench is a real-world verilog design benchmark, which focus on large verilog generation testing & hierarchical module design.
- It is much more complicate compared to verilog-eval/RTLLM
- It feature with a large, detailed spec that can help understand what should implement.
- real-world 4 series of Module (AES, E203, SDC) is more close to real-world demand

Benchmarks	Designs	Design Quality			Specification Quality		Verification Quality	
		Code Lines	Circuit Cells	SubModules	Doc. Lines	Multi-modal	Line Coverage	Formal
Thakur et al.	17	16.6	7.9	0	1.8	✗	100.0%	✗
RTLLMV1.1	29	56.1	48.1	0.3	22.3	✗	98.5%	✗
RTLLMV2	50	46.1	33.7	0.2	20.8	✗	96.0%	✗
VerilogEval-human	156	15.8	31.4	0	5.7	✗	99.7%	✗
VerilogEval-machine	143	13.9	6.4	0	3.2	✗	99.7%	✗
VerilogEvalV2	156	16.1	31.4	0	16.5	✗	99.7%	✗
ChipGPTV	33	51.2	18.0	0.6	12.5	✓	96.9%	✗
RealBench-module (ours)	60	241.2	0.52K	1.6	197.3	✓	100.0%	✓
RealBench-system (ours)	4	3537.3	1.1M	14.5	2.9k	✓	100.0%	✓

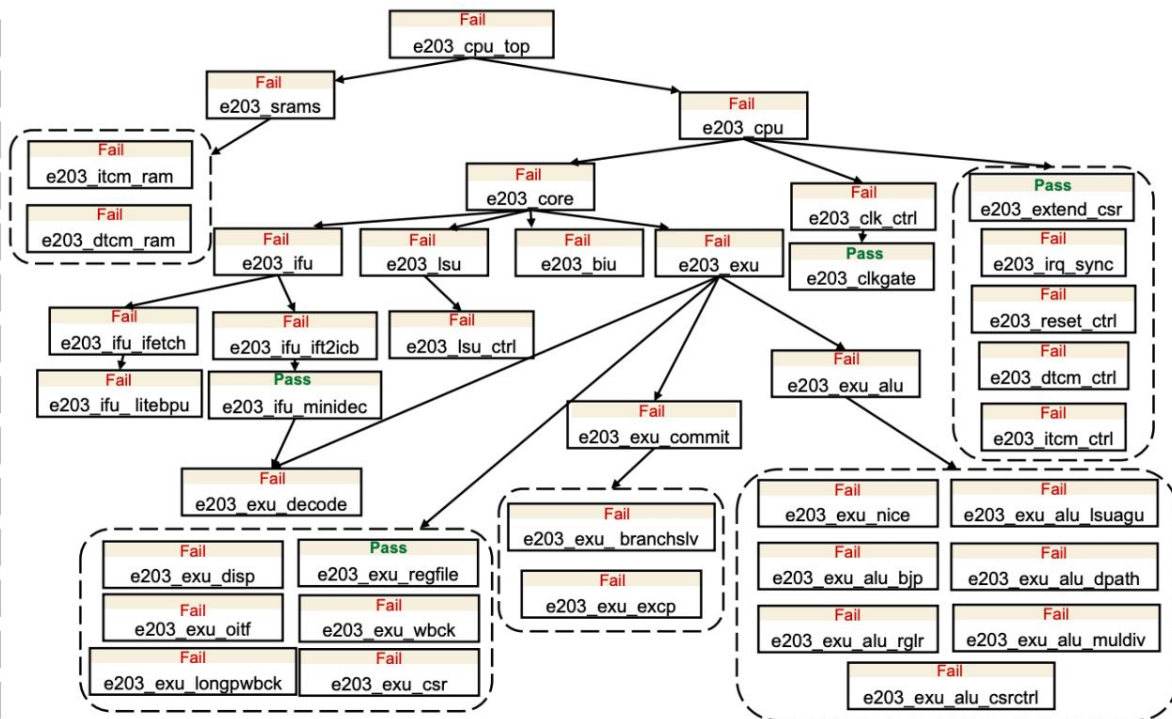
RealBench Structure



(a)



(b)



(c)

Realbench's Design hierarchy

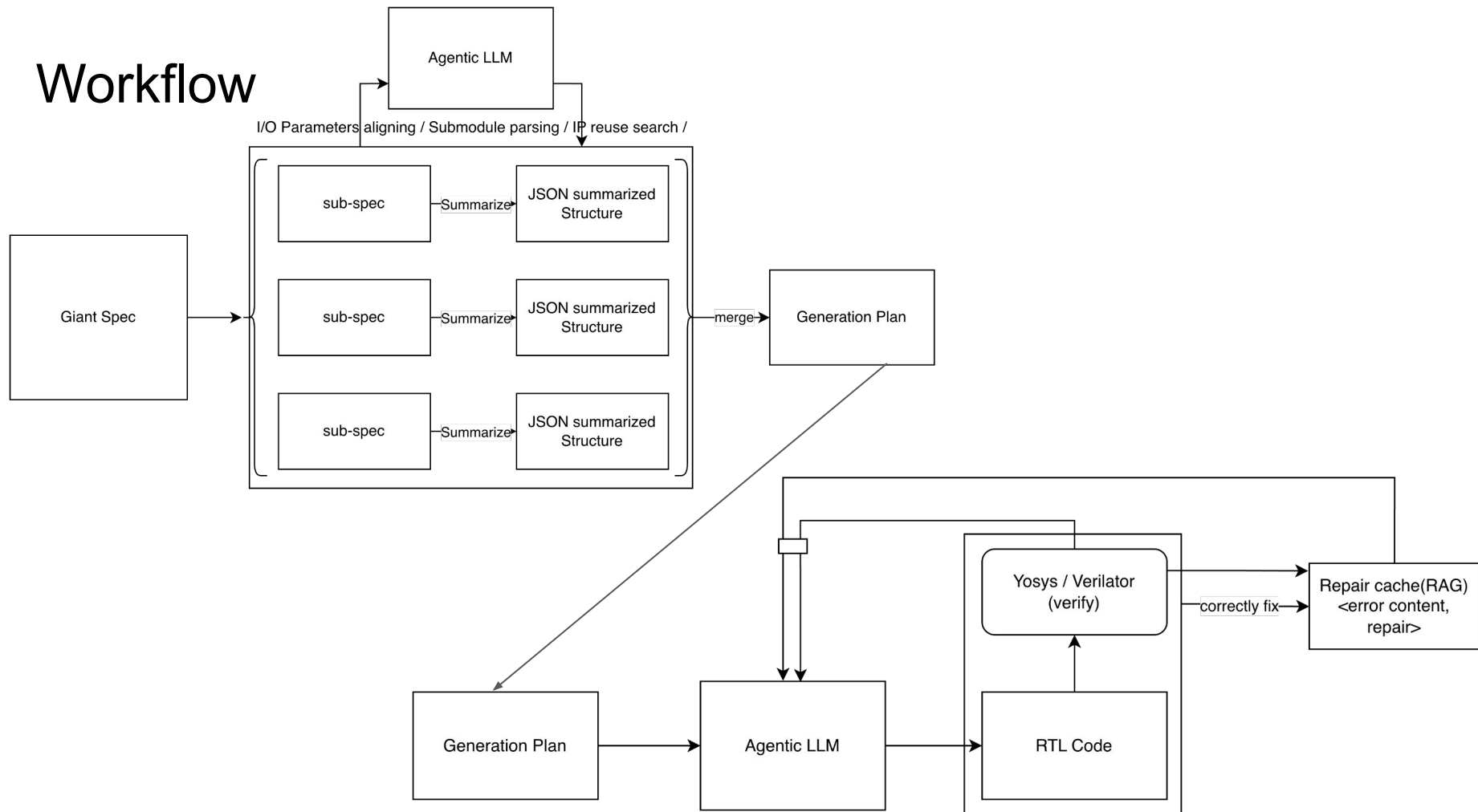
Outline

- Last time Action item: how RAG samples affect generation
- Feature of a hierarchical hardware design
- Realbench
- **agentic IP-reuse RTL generation pipeline Prototype**
 - Planning
 - Generate & fix (with RAG)
- **Experiment**
 - pure model v.s. prototype
 - issue & fix
 - v2 pipeline
 - repair cache ablation
 - error analysis
 - capability test
 - system level test
- **Action Items**

Full pipeline

- agentic planning
 - chunk summarize & merge
 - Sub-module decomposing
 - IP reuse Searching
- RTL gen & repair
 - Syntax verification
 - Repair Cache
 - Similarity RAG (not really use on RealBench since problem's uniqueness)

Workflow



Chunk Summarizing & merge

- Split the spec into several chunk, and ask LLM for summarizing the content
- Use a JSON format to store the keypoints
- focus on extracting the information including:
 - functionality, performance target, ppa constraint
 - some assumptions
 - submodules' information.

Tool Calling for IP reuse

Agentic-based method tends to let LLM call the relative tools to helps planning & IP reusing.

Tool	Role in the method
<code>search_reuse_ip</code>	Searches the per-task reusable-IP catalog for candidates.
<code>inspect_reuse_ip</code>	Returns behavior, interfaces, parameters, limits, and integration notes for one candidate.
<code>evaluate_ip_candidate</code>	Scores a candidate using criteria such as function match, interface compatibility, configurability, verification status, license, synthesis support, and documentation quality.
<code>write_artifact</code>	Writes Markdown or JSON artifacts into the plan output directory.
<code>generate_rtl_module</code>	Writes a complete synthesizable module when planning requires new RTL.
<code>validate_verilog</code>	Lints generated module files.
<code>check_port_compatibility</code>	Parses port declarations and verifies selected module connections.

Submodule Decomposing

- For a given module, we will try to decompose it into several sub-modules, including the module that mentioned in spec, but also some additional small sub-module split by LLM.
- For each module, we will retrieve the RAG to get IP list, with high similarity threshold, we can tag it as “module reuse” and give the interface information to top-module.
- If no suitable sub-module pass the threshold, the submodule will be marked as “new_rtl_required”, which makes LLM to generate a new sub-module RTL code.

Similarity RAG Reuse

- The pipeline module of the original semantic cache, tends to reduce the generation waste for similar module.
- But in current Realbench-based pipeline I disabled this mechanism since it might affect the accuracy of benchmark testing, which is unfair.
- But the module still can be used in usual scenario to speed-up/dehallucinate the generation process.

Code Repair Cache

- Current code generation loop will recursively run “code generate -> syntax verify -> repair if error”. and we notice that some similar error is often occur from different problems.
- To state the issue, we build a signature normalized encoder to map the same type of failure together.
- for example: After stripping identifiers and locations, two diagnostics from different root causes (a missing **wb_clk_i** pin in one module, a missing **core_clk** in another) become the same key
- We implement two types of cache:
 - run: all verilog task shared a same repair cache
 - task: only same task shared the same repair cache
 - The **run** version will be the first priority test subject since it more fit the real-world scenario.

Tree Decomposition (deprecated)

- There's a currently deprecated module call tree decomposition, is the initial idea of spec decomposing.
- It mainly focus on decomposed the module into submodule based on the give spec prompt.
- the decomposition process can be recursive, which build into a hierarchical tree structure which root is top module.
- And to maintain the generation process, we build a tree graph to store each sub-module's state, and generate the RTL code/reuse the IP buttom-up.
- But we face the problem that the decomposition process is usually too aggressive so that a simple module might also decompose with 3 or 4 layer of hierarchy. Which makes the generation more complicated and also raise the cost. So we don't really use this module in out pipeline now.

Outline

- Last time Action item: how RAG samples affect generation
- Feature of a hierarchical hardware design
- Realbench
- agentic IP-reuse RTL generation pipeline **Prototype**
 - Planning
 - Generate & fix (with RAG)
- **Experiment**
 - pure model v.s. prototype
 - issue & fix
 - v2 pipeline
 - repair cache ablation
 - error analysis
 - capability test
 - system level test
- Action Items

Experiment Setup

- Hardware:
 - Inference on 4*H100
- Model:
 - GPT-oss20B
 - GPT-oss120B
- Evaluation Benchmark:
 - RealBench modules set.
 - 20 samples for each questions.
- Metrics:
 - Syntax & Function on Pass@k, $k = 1, 5$

Pure Model acc. vs Plan&Gen Prototype

- can see that with the pipeline, both syntax pass rate & function pass rate has raise, which means the plan & gen pipeline helps LLM understanding the requirements and structure of the given module spec.

	Syntax		Function	
	Pass@1	Pass@5	Pass@1	Pass@5
Pure model	23.33	39.22	9.4	17.88
Prototype	61.08	85.39	17.92	27.16

prototype question accuracy breakdown

- [report](#)
- Instead of some large & complex problem, most of the question can be successfully pass the syntax correctness.
- Can see that the functional correct concentrated at a few question. Most of the problem maintain 0 functional pass.

Accurate Diff

- With the plan & generate pipeline, we can get the benefit of understanding the structurally-hard E203 blocks (e203_srams, e203_itcm_ram, e203_dtcn_ram, e203_exu_regfile, e203_exu_oitf, e203_exu_alu_csctrl)
- However, after compare the Pass content, we notice that some of the question only pass on the pure model generate test.
 - They are mostly on AES & SDC question set.

module	pipeline syntax	pipeline function	direct syntax	direct function
aes_sbox	18/20	0	17/20	4
aes_inv_cipher_top	16/20	0	14/20	12
sd_cmd_serial_host	18/20	0	15/20	12
sd_tx_fifo	18/20	0	9/20	9

Accurate Diff

- Since our repair loop only check for the syntax correctness, so it optimizes a syntax objective, which might rescues functionally-wrong code into the compiles bucket without making it correct.
- Break-down:
 - **Direct** generate solve 26 distinct module, mostly on AES & SDC, and most rely on the exploration of the solution space. (1 pass in 20 samples) Mean 6.8 / 20 passing each.
 - **Pipeline** solve 19 distinct module, more pass on E203 module, and can stably pass the same module. Mean 12.7 / 20 passing each.
 - Direct generate have wider coverage but lower precision.
 - Pipeline have narrow coverage but higher precision.

Some problem on prototype & fix

- agentic tool use can't be efficiently call by LLM
 - for 60 questions * 20 samples, 0 tool calls try, model is too confidence on answering
 - Solution: a parser that get and force injecting submodule content into prompt
- While planning, model usually occur some hallucination problems (parameters name slightly changed, sub-module name miss align.)
 - also because of agentic tool calling doesn't execute successfully
 - Solution: after planning, we'll verify the sub-module content in plan with actual submodule information to avoid hallucination.

Accuracy After Fix (v2)

- Pass@1 raise means the modification that inject the submodule helps understanding the structure of module.
- But Pass@5 stay the same means that it still can't fix those incorrect questions(pass 0/20 at first).
- So we may set the tone of this version as “help LLM generate more precise code for a solvable question.”

	Syntax		Function	
	Pass@1	Pass@5	Pass@1	Pass@5
Prototype	61.08	82.39	17.92	27.16
Fixed version	66.92	83.75	20.17	27.40

Repair cache compare (run/task/off)

- Can see that the run version have highest pass rate (slightly)
- but with repair cache, the repair process can be substantially speed-up since no need to reasoning again to find the problem.

	Syntax Pass Rate	Function Pass Rate	Cache Hit Rate
run	66.9%	20.2%	236/281 = 84%
task	64.3%	19.6%	63/402 = 16%
off	59.4%	18.5%	X

	Mean sample latency	total sample latency	speed-up
run	230 s	4238 s	3.09x
task	419 s	5651 s	2.31x
off	1272 s	13099 s	1.0x

Error analysis

- The outcome split:

Bucket	Count	Share
Pass (syntax + function)	242	20.2%
Compiles but function wrong	561	46.8%
Syntax fail	384	32.0%
Not generated	13	1.1%

- Syntax error distribution:

Class	Recs	%	What it is
MISSING-MODULE	97	25.3%	instantiates a submodule never declared
PINNOTFOUND	77	20.1%	module lacks a port the testbench drives — giant DUTs
PARSE	64	16.7%	raw malformed Verilog
DUPDECL	53	13.8%	duplicate signal/port declaration
IMPLICITSTATIC	33	8.6%	static init in non-static scope (co-occurs)
CONNECT-VAR	28	7.3%	connects an instance pin to a variable, not a net
ASSIGNIN	27	7.0%	assignment to an input/net
PARAM-UNRESOLVED	20	5.2%	can't resolve a param at instantiation
NOTCONST	20	5.2%	non-constant where constant required (undefined macro/param)
ALWNEVER / ZERODLY	19 / 19	4.9%	always never triggers / #0 delay
DUPPIN	18	4.7%	duplicate pin connection
misc	≤8 each		misc

Error analysis

- Function error distribution: need to notice that function error means there's some output mismatch at some timing.
 - So we can categorize with two ways: First mismatch & how many ports are wrong.
- 62.6% of error is whole I/O behavior error so that more than 4 outputs are wrong
- Those late stage error usually because of real functional misunderstanding, a functional-feedback repair turn could possibly rescue.

	Records	%
4+ outputs wrong (broad)	351	62.6%
1 output wrong	117	20.9%
2–3 outputs	93	16.6%

	Records	%
Early ($t \leq 20$, ~reset/initial behavior)	324	59.7%
Late ($t > 20$, sequential divergence)	219	40.3%

Error Analysis

- Error distribution across each question set

Family	Pass	Func-fail	Syntax-fail	Not-gen
e203_hbirdv2	158	315	314	13
sdc	59	158	63	0
aes	25	88	7	0

- Most syntax failure occur on E203, which contain most of large interconnect/integrator modules.
- Most of AES samples fail at functional test.
- For 13 not generated samples, 6 of them is because of context overflow.

Furthur Fix from Error Analysis

- For syntax error, we might need to implement
 - sub-agent partial code generation and merge
 - or more aggressive summarize/context filter methods
- For functional error, we should include the testbench check into the gen-repair loop
 - which is reasonable since IC designer also need to interact with testbench to ensure the RTL code is correct, which verification should be a part of generation, not only for result evaluation.

Capability Test

- oss 20B v.s. oss 120B
- Both run on v2 pipeline

	Syntax		Function	
	Pass@1	Pass@5	Pass@1	Pass@5
oss 20B	66.92	83.75	20.17	27.40
oss 120B	76.33	89.49	24.50	32.76

- the accuracy raise come from 2 more successful implementation of E203 module.(e203_exu_alu_bjp & e203_exu_commit)

System Problem Test

- Evaluate 5 samples of each system level problem using oss 120B
- Need to notice that all functional correctness is 0.
- Current chunk & merge planning can help process long spec to generate a initial syntax correct code, but still far from correct.
- Super long spec (e203_cpu_top) is still hard to successfully been process.

	spec character amount	Syntax pass	direct generate pass
aes_chiper_top	27469	5/5	1/5
aes_inv_chiper_top	39997	5/5	3/5
sdc_controller	225146	3/5	0/5
e230_cpu_top	703808	0/5	0/5

Outline

- Last time Action item: how RAG samples affect generation
- Feature of a hierarchical hardware design
- Realbench
- agentic IP-reuse RTL generation pipeline **Prototype**
 - Planning
 - Generate & fix (with RAG)
- **Experiment**
 - pure model v.s. prototype
 - issue & fix
 - v2 pipeline
 - repair cache ablation
 - error analysis
 - capability test
 - system level test
- **Action Items**

Action experiments

- Repair with spec as reference
 - improve repair loop
- A repair loop based on testbench (currently only verilator/yosys syntax check)
 - improve functional verifying
- Similarity RAG reuse experiment to test the speed-up
 - inference QoS improvement
- Model Capability Experiment
 - DeepSeek V3/R1 (large base model)
 - CodeV-R1, SiliconMind-V1 (verilog specific Model: pure model or with pipeline)
 - MAGE, VerilogCoder (other verilog coding methods comparison)
- Explore a suitable method to make the tree decomposition method re-join the pipeline
 - make problem divide&conquerable, may also help super large spec processing

06/23 update

research of IP reuse on realbench

Recap

last time we build a multi-level retrieval and caching assistance for RTL coding agent, and test on verilog-Eval & RTLLM.

And to embody the value of this, we want to apply it on a real-world IP reuse scenario now.

Outline

- **Last time Action item: how RAG samples affect generation**
- Feature of a hierarchical hardware design
- Realbench
- agentic IP-reuse RTL generation pipeline **Prototype**
 - Planning
 - Generate & fix (with RAG)
- **Experiment**
 - pure model v.s. prototype
 - issue & fix
 - v2 pipeline
 - repair cache ablation
 - error analysis
 - capability test
 - system level test
- **Action Items**

Benchmark test intro

- RAG is a **outer information support system** designed to help the LLM's generation to **avoid from hallucination** and supplement sufficient knowledge.
- Which means for a verilog generation task, if use RAG, then it should supply a similar enough sample as reference or even give the result directly.
- Also for a real world verilog coding spec, the hierarchical module structure more common rather than a simple single module hardware.
- When decomposing a complex module into several submodules, the following situation may occur: several modules may be **built from the same submodule**.
- If we can correctly reuse this module (or IP), we can accelerate the generation process and save lots of resources.

Benchmark test

- For those correct, does RAG get same document?
 - determine that model can pass the test is because of it's own ability or the correct reference
- For those incorrect, what RAG retrieve, how similar is it?
 - determine low quality reference will affect result or not.
- So we check the generation result of siliconMind-v1 with **pure model & rag mode**.
- In case study, we catagorize the error type of each generation fail as following:
 - S = Syntax Error
 - C = Compile Error
 - F = Functional Error
 - G = Generate Error (no code been parsed)
 - R = Runtime Failure

Benchmark test

- Prob028_m2014_q4a

	Pass Rate	Retrieve Similarity
Pure Model	40%, 2S1C	X
RAG Mode	80%, 1C	0.885

I would like you to implement a module named TopModule with the following interface. All input and output ports are one bit unless otherwise specified.

- input d
- input ena
- output q

The module should implement a **D latch** using an always block.

```
`default_nettype none

module top_module (
    input d,
    input ena,
    output reg q
);
    always @(*) begin
        if (ena) q = d;
    end
endmodule
```

Design a level-sensitive **D latch** in Verilog with the following specifications:

****Module name:**** `top\_module`
****Port list (exact order and widths):****

- `input d` : single-bit data input.
- `input ena` : single-bit enable (gate) input. When `ena` is high the latch should be transparent, i.e., the output follows `d`. When `ena` is low the latch must hold its previous value.
- `output reg q` : single-bit latched output.

* The latch must be described using a single combinational `always @(\*)` block. Inside this block, assign `q` only when `ena` is high (use a blocking assignment). Do ****not**** provide an explicit `else` branch; the absence of an assignment when `ena` is low should cause a latch to be inferred that retains its state.

* Include the compiler directive `` `default\_nettype none `` before the module declaration to require all nets to be explicitly declared.

Write the complete Verilog code that satisfies these requirements. The code you produce must be exactly the one shown in the snippet below.

Benchmark test

- [Prob150_review2015_fsmonehot](#)
 - The question is ask for a **combinational part of a one-hot encoded Moore FSM**.
- [Retrieve document](#)
 - The question asks for a **complete FSM controller**, which mixed Moore & Mealy FSM
- And there's also many different detail
 - clk, reset, even I/O is very different.

	Pass Rate	Retrieve Similarity
Pure Model	100%	X
RAG Mode	0%, 3R2G	0.614

I would like you to implement a module named TopModule with the following interface. All input and output ports are one bit unless otherwise specified.

- input d
- input done_counting
- input ack
- input state (10 bits)
- output B3_next
- output S_next
- output S1_next
- output Count_next
- output Wait_next
- output done
- output counting
- output shift_ena

The module should implement the following Moore state machine with 3 input (d, done_counting, ack) and 3 outputs (shift_ena, counting, done). Unless otherwise stated in the diagram below, assume outputs are 0 and inputs are don't cares.

state	(output)	--input--> next state
-------	----------	-----------------------

S	()	--d=0--> S
S	()	--d=1--> S1
S1	()	--d=0--> S
S1	()	--d=1--> S11
S11	()	--d=0--> S110
S11	()	--d=1--> S11
S110	()	--d=0--> S
S110	()	--d=1--> B0
B0	(shift_ena=1)	--(always go to next cycle)--> B1
B1	(shift_ena=1)	--(always go to next cycle)--> B2
B2	(shift_ena=1)	--(always go to next cycle)--> B3
B3	(shift_ena=1)	--(always go to next cycle)--> Count
Count	(counting=1)	--done_counting=0--> Count
Count	(counting=1)	--done_counting=1--> Wait
Wait	(done=1)	--ack=0--> Wait
Wait	(done=1)	--ack=1--> S

At reset, the state machine starts in state "S". Derive next-state logic equations and output logic equations by inspection assuming the following one-hot encoding is used: (S, S1, S11, S110, B0, B1, B2, B3, Count, Wait) = (10'b0000000001, 10'b0000000010, 10'b00000000100, ... , 10'b1000000000)

Derive state transition and output logic equations by inspection assuming a one-hot encoding. Implement only the state transition logic and output logic (the combinational logic portion) for this state machine.

Write code that generates the following signals:

- B3_next -- Assert when next-state is B3 state
- S_next -- Assert when next-state is S state
- S1_next -- Assert when next-state is S1 state
- Count_next -- Assert when next-state is Count state
- Wait_next -- Assert when next-state is Wait state
- done -- output logic
- counting -- output logic
- shift_ena -- output logic

Design the control unit for a **sequential binary multiplier** using the **one-flip-flop-per-state** coding technique.

The controller must manage three distinct phases of the multiplication process:

- Idle (ready)** – The multiplier is waiting for a new operation.
- Load (decr_p)** – The multiplicand and multiplier registers are loaded.
- Shift/Multiply (shift_regs)** – The registers are repeatedly shifted and, when appropriate, the partial product is added.

The required behaviour is:

- The FSM is reset to the idle state when the active-low synchronous reset (``reset``) is asserted.
- While in the idle state (``ready`` = 1), asserting the ``start`` input should move the FSM to the **load** state.
- After one clock cycle in the load state, the FSM should automatically advance to the **shift** state.
- The FSM stays in the shift state until the input ``zero`` becomes high, indicating that the multiplication is finished; at that moment it should return to the idle state on the next clock edge.
- Moore outputs** (``ready``, ``decr_p``, ``shift_regs``) must reflect the present state only.
- Mealy outputs**:
 - ``load_regs`` must be high when the controller is in the idle state **and** ``start`` is high (i.e., the moment the multiplication begins).
 - ``add_regs`` must be high when the controller is in the load state **and** the least-significant bit of the multiplier (``Q0``) is high (i.e., a partial-product addition should occur).

Implement the controller as a Verilog module named **control_multiplier_2** with the following port list:

Signal	Direction	Width	Description
<code>`start`</code>	input	1	Starts a new multiplication
<code>`zero`</code>	input	1	Indicates that the product has become zero (end of multiplication)
<code>`Q0`</code>	input	1	LSB of the multiplier operand (used for addition)
<code>`clk`</code>	input	1	System clock (positive edge)
<code>`reset`</code>	input	1	Active-low synchronous reset
<code>`ready`</code>	output	1	Moore output – high in idle state
<code>`decr_p`</code>	output	1	Moore output – high in load state
<code>`shift_regs`</code>	output	1	Moore output – high in shift state
<code>`load_regs`</code>	output	1	Mealy output – high when <code>`ready`</code> & <code>`start`</code>
<code>`add_regs`</code>	output	1	Mealy output – high when <code>`decr_p`</code> & <code>`Q0`</code>

Use primitive gate primitives (``and``, ``or``, ``buf``) for the combinational next-state logic and a single ``always @(posedge clk)`` block for the sequential state register. The reset should be handled synchronously inside the clocked block (i.e., ``if (!reset) state <= 3'b000;``).

<pre> module control_multiplier_2(input start, input zero, input Q0, input clk, input reset, output ready, output decr_p, output shift_regs, output load_regs, output add_regs); reg [2:0] state; wire ns0, ns1, ns2; wire s0_not_start, s2_zero, s0_start, s2_not_zero; and(s0_not_start, ~start, state[0]); and(s2_zero, zero, state[2]); or (ns0, s0_not_start, s2_zero); and(s0_start, state[0], start); buf(ns1, s0_start); </pre>	<pre> and(s2_not_zero, ~zero, state[2]); or (ns2, state[1], s2_not_zero); always @(posedge clk) begin if (!reset) state <= 3'b001; else begin state[0] <= ns0; state[1] <= ns1; state[2] <= ns2; end end assign ready = state[0]; assign decr_p = state[1]; assign shift_regs = state[2]; and(load_regs, state[0], start); and(add_regs, state[1], Q0); endmodule </pre>
---	---

Benchamrk Test - Conclusion

- So we can see that, a good retrieve sample can help LLM to generate a correct result.
- But also a bad retrieve sample may mislead the model to become confused about what we're going to do.
- Conclusion: Add a **hard similarity threshold** for retrieved sample, only reference these samples when it is good/similar enough to help understanding the question. Else should fallback to generate by LLM itself.

Outline

- Last time Action item: how RAG samples affect generation
- **Feature of a hierarchical hardware design**
- Realbench
- agentic IP-reuse RTL generation pipeline **Prototype**
 - Planning
 - Generate & fix (with RAG)
- **Experiment**
 - pure model v.s. prototype
 - issue & fix
 - v2 pipeline
 - repair cache ablation
 - error analysis
 - capability test
 - system level test
- **Action Items**

IP reuse feature & Solution

1. Spec is usually too large to be process in one-shot (700k characters)
2. Current verilog gen model can hard to directly generate the correct code.

As for solution:

1. Use a large context orchestrator do context catagorize and splits into multiple document for multiple agent parallel code generation.
2. Build a hierarchical graph structure to store the state of decomposition module and trace for code build.

Outline

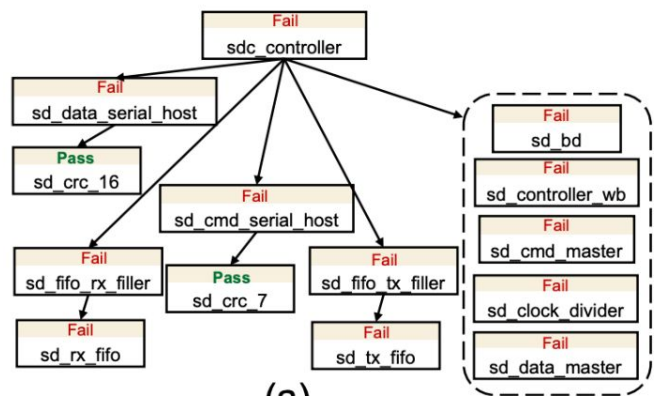
- Last time Action item: how RAG samples affect generation
- Feature of a hierarchical hardware design
- **Realbench**
- agentic IP-reuse RTL generation pipeline **Prototype**
 - Planning
 - Generate & fix (with RAG)
- **Experiment**
 - pure model v.s. prototype
 - issue & fix
 - v2 pipeline
 - repair cache ablation
 - error analysis
 - capability test
 - system level test
- **Action Items**

RealBench^[1]

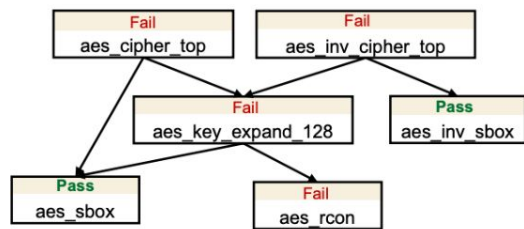
- RealBench is a real-world verilog design benchmark, which focus on large verilog generation testing & hierarchical module design.
- It is much more complicate compared to verilog-eval/RTLLM
- It feature with a large, detailed spec that can help understand what should implement.
- real-world 4 series of Module (AES, E203, SDC) is more close to real-world demand

Benchmarks	Designs	Design Quality			Specification Quality		Verification Quality	
		Code Lines	Circuit Cells	SubModules	Doc. Lines	Multi-modal	Line Coverage	Formal
Thakur et al.	17	16.6	7.9	0	1.8	✗	100.0%	✗
RTLLMV1.1	29	56.1	48.1	0.3	22.3	✗	98.5%	✗
RTLLMV2	50	46.1	33.7	0.2	20.8	✗	96.0%	✗
VerilogEval-human	156	15.8	31.4	0	5.7	✗	99.7%	✗
VerilogEval-machine	143	13.9	6.4	0	3.2	✗	99.7%	✗
VerilogEvalV2	156	16.1	31.4	0	16.5	✗	99.7%	✗
ChipGPTV	33	51.2	18.0	0.6	12.5	✓	96.9%	✗
RealBench-module (ours)	60	241.2	0.52K	1.6	197.3	✓	100.0%	✓
RealBench-system (ours)	4	3537.3	1.1M	14.5	2.9k	✓	100.0%	✓

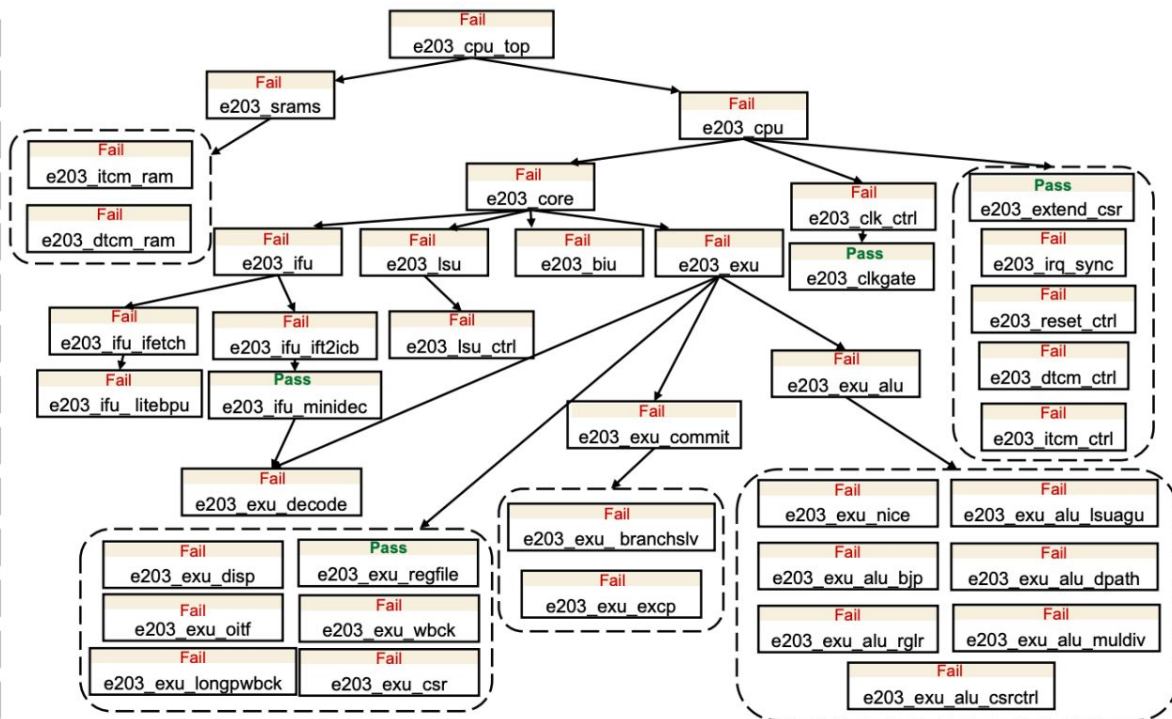
RealBench Structure



(a)



(b)



(c)

Realbench's Design hierarchy

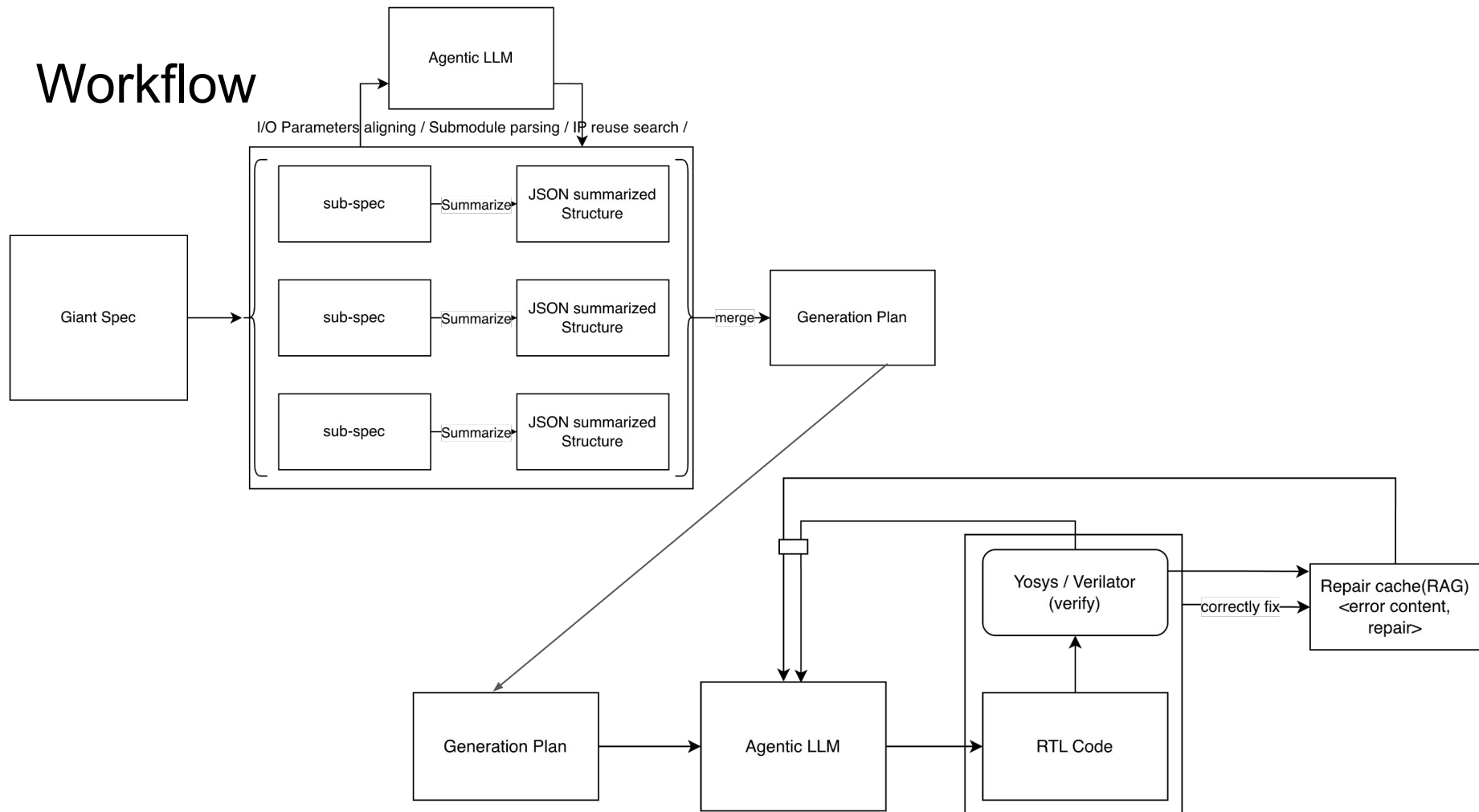
Outline

- Last time Action item: how RAG samples affect generation
- Feature of a hierarchical hardware design
- Realbench
- **agentic IP-reuse RTL generation pipeline Prototype**
 - Planning
 - Generate & fix (with RAG)
- **Experiment**
 - pure model v.s. prototype
 - issue & fix
 - v2 pipeline
 - repair cache ablation
 - error analysis
 - capability test
 - system level test
- **Action Items**

Full pipeline

- agentic planning
 - chunk summarize & merge
 - Sub-module decomposing
 - IP reuse Searching
- RTL gen & repair
 - Syntax verification
 - Repair Cache
 - Similarity RAG (not really use on RealBench since problem's uniqueness)

Workflow



Chunk Summarizing & merge

- Split the spec into several chunk, and ask LLM for summarizing the content
- Use a JSON format to store the keypoints
- focus on extracting the information including:
 - functionality, performance target, ppa constraint
 - some assumptions
 - submodules' information.

Tool Calling for IP reuse

Agentic-based method tends to let LLM call the relative tools to helps planning & IP reusing.

Tool	Role in the method
<code>search_reuse_ip</code>	Searches the per-task reusable-IP catalog for candidates.
<code>inspect_reuse_ip</code>	Returns behavior, interfaces, parameters, limits, and integration notes for one candidate.
<code>evaluate_ip_candidate</code>	Scores a candidate using criteria such as function match, interface compatibility, configurability, verification status, license, synthesis support, and documentation quality.
<code>write_artifact</code>	Writes Markdown or JSON artifacts into the plan output directory.
<code>generate_rtl_module</code>	Writes a complete synthesizable module when planning requires new RTL.
<code>validate_verilog</code>	Lints generated module files.
<code>check_port_compatibility</code>	Parses port declarations and verifies selected module connections.

Submodule Decomposing

- For a given module, we will try to decompose it into several sub-modules, including the module that mentioned in spec, but also some additional small sub-module split by LLM.
- For each module, we will retrieve the RAG to get IP list, with high similarity threshold, we can tag it as “module reuse” and give the interface information to top-module.
- If no suitable sub-module pass the threshold, the submodule will be marked as “new_rtl_required”, which makes LLM to generate a new sub-module RTL code.

Similarity RAG Reuse

- The pipeline module of the original semantic cache, tends to reduce the generation waste for similar module.
- But in current Realbench-based pipeline I disabled this mechanism since it might affect the accuracy of benchmark testing, which is unfair.
- But the module still can be used in usual scenario to speed-up/dehallucinate the generation process.

Code Repair Cache

- Current code generation loop will recursively run “code generate -> syntax verify -> repair if error”. and we notice that some similar error is often occur from different problems.
- To state the issue, we build a signature normalized encoder to map the same type of failure together.
- for example: After stripping identifiers and locations, two diagnostics from different root causes (a missing **wb_clk_i** pin in one module, a missing **core_clk** in another) become the same key
- We implement two types of cache:
 - run: all verilog task shared a same repair cache
 - task: only same task shared the same repair cache
 - The **run** version will be the first priority test subject since it more fit the real-world scenario.

Tree Decomposition (deprecated)

- There's a currently deprecated module call tree decomposition, is the initial idea of spec decomposing.
- It mainly focus on decomposed the module into submodule based on the give spec prompt.
- the decomposition process can be recursive, which build into a hierarchical tree structure which root is top module.
- And to maintain the generation process, we build a tree graph to store each sub-module's state, and generate the RTL code/reuse the IP buttom-up.
- But we face the problem that the decomposition process is usually too aggressive so that a simple module might also decompose with 3 or 4 layer of hierarchy. Which makes the generation more complicated and also raise the cost. So we don't really use this module in out pipeline now.

Outline

- Last time Action item: how RAG samples affect generation
- Feature of a hierarchical hardware design
- Realbench
- agentic IP-reuse RTL generation pipeline **Prototype**
 - Planning
 - Generate & fix (with RAG)
- **Experiment**
 - pure model v.s. prototype
 - issue & fix
 - v2 pipeline
 - repair cache ablation
 - error analysis
 - capability test
 - system level test
- Action Items

Experiment Setup

- Hardware:
 - Inference on 4*H100
- Model:
 - GPT-oss20B
 - GPT-oss120B
- Evaluation Benchmark:
 - RealBench modules set.
 - 20 samples for each questions.
- Metrics:
 - Syntax & Function on Pass@k, $k = 1, 5$

Pure Model acc. vs Plan&Gen Prototype

- can see that with the pipeline, both syntax pass rate & function pass rate has raise, which means the plan & gen pipeline helps LLM understanding the requirements and structure of the given module spec.

	Syntax		Function	
	Pass@1	Pass@5	Pass@1	Pass@5
Pure model	23.33	39.22	9.4	17.88
Prototype	61.08	85.39	17.92	27.16

prototype question accuracy breakdown

- [report](#)
- Instead of some large & complex problem, most of the question can be successfully pass the syntax correctness.
- Can see that the functional correct concentrated at a few question. Most of the problem maintain 0 functional pass.

Accurate Diff

- With the plan & generate pipeline, we can get the benefit of understanding the structurally-hard E203 blocks (e203_srams, e203_itcm_ram, e203_dtcn_ram, e203_exu_regfile, e203_exu_oitf, e203_exu_alu_csctrl)
- However, after compare the Pass content, we notice that some of the question only pass on the pure model generate test.
 - They are mostly on AES & SDC question set.

module	pipeline syntax	pipeline function	direct syntax	direct function
aes_sbox	18/20	0	17/20	4
aes_inv_cipher_top	16/20	0	14/20	12
sd_cmd_serial_host	18/20	0	15/20	12
sd_tx_fifo	18/20	0	9/20	9

Accurate Diff

- Since our repair loop only check for the syntax correctness, so it optimizes a syntax objective, which might rescues functionally-wrong code into the compiles bucket without making it correct.
- Break-down:
 - **Direct** generate solve 26 distinct module, mostly on AES & SDC, and most rely on the exploration of the solution space. (1 pass in 20 samples) Mean 6.8 / 20 passing each.
 - **Pipeline** solve 19 distinct module, more pass on E203 module, and can stably pass the same module. Mean 12.7 / 20 passing each.
 - Direct generate have wider coverage but lower precision.
 - Pipeline have narrow coverage but higher precision.

Some problem on prototype & fix

- agentic tool use can't be efficiently call by LLM
 - for 60 questions * 20 samples, 0 tool calls try, model is too confidence on answering
 - Solution: a parser that get and force injecting submodule content into prompt
- While planning, model usually occur some hallucination problem(parameters name slightly changed, sub-module name miss align.)
 - also because of agentic tool calling doesn't execute successfully
 - Solution: after planning, we'll verify the sub-module content in plan with actual submodule information to avoid hallucination.

Accuracy After Fix (v2)

- Pass@1 raise means the modification that inject the submodule helps understanding the structure of module.
- But Pass@5 stay the same means that it still can't fix those incorrect question(pass 0/20 at first).
- So we may set the tone of this version as “help LLM generate more precise code for a solvable question.”

	Syntax		Function	
	Pass@1	Pass@5	Pass@1	Pass@5
Prototype	61.08	82.39	17.92	27.16
Fixed version	66.92	83.75	20.17	27.40

Repair cache compare (run/task/off)

- Can see that the run version have highest pass rate (slightly)
- but with repair cache, the repair process can be substantially speed-up since no need to reasoning again to find the problem.

	Syntax Pass Rate	Function Pass Rate	Cache Hit Rate
run	66.9%	20.2%	236/281 = 84%
task	64.3%	19.6%	63/402 = 16%
off	59.4%	18.5%	X

	Mean sample latency	total sample latency	speed-up
run	230 s	4238 s	3.09x
task	419 s	5651 s	2.31x
off	1272 s	13099 s	1.0x

Error analysis

- The outcome split:

Bucket	Count	Share
Pass (syntax + function)	242	20.2%
Compiles but function wrong	561	46.8%
Syntax fail	384	32.0%
Not generated	13	1.1%

- Syntax error distribution:

Class	Recs	%	What it is
MISSING-MODULE	97	25.3%	instantiates a submodule never declared
PINNOTFOUND	77	20.1%	module lacks a port the testbench drives — giant DUTs
PARSE	64	16.7%	raw malformed Verilog
DUPDECL	53	13.8%	duplicate signal/port declaration
IMPLICITSTATIC	33	8.6%	static init in non-static scope (co-occurs)
CONNECT-VAR	28	7.3%	connects an instance pin to a variable, not a net
ASSIGNIN	27	7.0%	assignment to an input/net
PARAM-UNRESOLVED	20	5.2%	can't resolve a param at instantiation
NOTCONST	20	5.2%	non-constant where constant required (undefined macro/param)
ALWNEVER / ZERODLY	19 / 19	4.9%	always never triggers / #0 delay
DUPPIN	18	4.7%	duplicate pin connection
misc	≤8 each		misc

Error analysis

- Function error distribution: need to notice that function error means there's some output mismatch at some timing.
 - So we can categorize with two ways: First mismatch & how many ports are wrong.
- 62.6% of error is whole I/O behavior error so that more than 4 outputs are wrong
- Those late stage error usually because of real functional misunderstanding, a functional-feedback repair turn could possibly rescue.

	Records	%
4+ outputs wrong (broad)	351	62.6%
1 output wrong	117	20.9%
2–3 outputs	93	16.6%

	Records	%
Early ($t \leq 20$, ~reset/initial behavior)	324	59.7%
Late ($t > 20$, sequential divergence)	219	40.3%

Error Analysis

- Error distribution across each question set

Family	Pass	Func-fail	Syntax-fail	Not-gen
e203_hbirdv2	158	315	314	13
sdc	59	158	63	0
aes	25	88	7	0

- Most syntax failure occur on E203, which contain most of large interconnect/integrator modules.
- Most of AES samples fail at functional test.
- For 13 not generated samples, 6 of them is because of context overflow.

Furthur Fix from Error Analysis

- For syntax error, we might need to implement
 - sub-agent partial code generation and merge
 - or more aggressive summarize/context filter methods
- For functional error, we should include the testbench check into the gen-repair loop
 - which is reasonable since IC designer also need to interact with testbench to ensure the RTL code is correct, which verification should be a part of generation, not only for result evaluation.

Capability Test

- oss 20B v.s. oss 120B
- Both run on v2 pipeline

	Syntax		Function	
	Pass@1	Pass@5	Pass@1	Pass@5
oss 20B	66.92	83.75	20.17	27.40
oss 120B	76.33	89.49	24.50	32.76

- the accuracy raise come from 2 more successful implementation of E203 module.(e203_exu_alu_bjp & e203_exu_commit)

System Problem Test

- Evaluate 5 samples of each system level problem using oss 120B
- Need to notice that all functional correctness is 0.
- Current chunk & merge planning can help process long spec to generate a initial syntax correct code, but still far from correct.
- Super long spec (e203_cpu_top) is still hard to successfully been process.

	spec character amount	Syntax pass	direct generate pass
aes_chiper_top	27469	5/5	1/5
aes_inv_chiper_top	39997	5/5	3/5
sdc_controller	225146	3/5	0/5
e230_cpu_top	703808	0/5	0/5

Outline

- Last time Action item: how RAG samples affect generation
- Feature of a hierarchical hardware design
- Realbench
- agentic IP-reuse RTL generation pipeline **Prototype**
 - Planning
 - Generate & fix (with RAG)
- **Experiment**
 - pure model v.s. prototype
 - issue & fix
 - v2 pipeline
 - repair cache ablation
 - error analysis
 - capability test
 - system level test
- **Action Items**

Action experiments

- Repair with spec as reference
 - improve repair loop
- A repair loop based on testbench (currently only verilator/yosys syntax check)
 - improve functional verifying
- Similarity RAG reuse experiment to test the speed-up
 - inference QoS improvement
- Model Capability Experiment
 - DeepSeek V3/R1 (large base model)
 - CodeV-R1, SiliconMind-V1 (verilog specific Model: pure model or with pipeline)
 - MAGE, VerilogCoder (other verilog coding methods comparison)
- Explore a suitable method to make the tree decomposition method re-join the pipeline
 - make problem divide&conquerable, may also help super large spec processing

update

Outline

- Hybrid Generate (Pipeline / Direct) Test
 - Simple Hybrid generate result
 - Attribute of pipeline-success set & direct success set
- New Functionality
 - Router Design
 - Functional Repair
 - Spec Slice & Injection
- Full Architecture
- Evaluation
 - v2 + Spec Slice & Functional Repair
 - Repair Distribution
 - Scale up Hybrid test
 - Router (Performance / Accuracy)

Benchmark Error

- For `aes_sbox` & `aes_inv_sbox`, there's a specific **false negative** mistake happens that, it **check the output immediately after the edge trigger** to change the input, since output usually need a little more time to be changed after input change.
- To fix this issue, we can add a `#1` delay to force the output check trigger at next clock, let the real output have it time to modify the the correct signal.

	Syntax Pass	Original Functional Pass	Functional Pass after Fix
<code>aes_sbox</code>	18/20	0/20	13/20
<code>aes_inv_sbox</code>	19/20	0/20	17/20

A Hybrid Test

- We combine the pipeline & direct version of generation type into a single run, run with **both 10 samples**, and compute the pass rate together/separately.
- We observe the same conclusion: direct generate have wider variety but less precise, pipeline generate have narrow variety but more precise.
- After combination, we can get up to 30 tasks successfully solve with following break-out:

Flow	Samples	Syntax	Func pass	Solved	pass@1	pass@5	pass@10
Pipeline	600	65.3%	20.0%	17/60	20.0%	27.3%	28.3%
Direct	600	26.3%	16.7%	24/60	16.7%	31.2%	40.0%
Combined (hybrid)	1200	45.8%	18.3%	30/60	18.3%	36.6%	43.0%

Hybrid Test Conclusion: Design A Router

- With the result, we know that pipeline & direct model have a different using scenario that we can actually get a higher performance if we combine them.
- So the next question is, can we build a **router** with a good strategy to distribute the question into sufficient workflow? First we need to evaluate the specific test different mode can achieve:
- **Pipeline only:** e203_dtcram, e203_exu_alu_csctrl, e203_exu_alu_rglr, e203_exu_regfile, e203_ifu, e203_ifu_minidec, e203_itcmram, e203_srams
- **Direct model only:** aes_cipher_top, aes_inv_cipher_top, e203_exu_branchslv, e203_exu_alu_lsquag, e203_exu_decode, sd_cmd_master, sd_cmd_serial_host, sd_rx_fifo, sd_tx_fifo.
- What's the difference between two sets of correct problem?

Attribute Differences

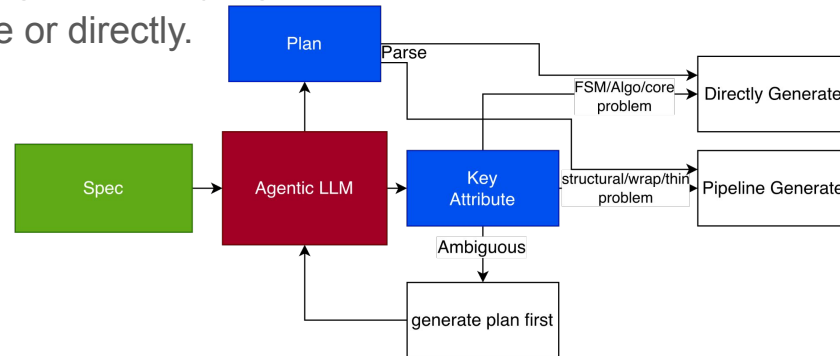
- We evaluate two sets' code lines, spec size, and the own cells(after compile)
- After careful analyze, we can conclude the characteristic of two sets:
 - **Pipeline mode:** can correctly process the task that mostly doing the integration of different module, build the interface & connection between each module. Which means it have good performance on structural / wrapper / thin-combinational question.
 - **Direct mode:** can **usually** correctly process the task that implementing a algorithm / stateful / datapath core problem.

指標(中位數 / 平均 / 範圍)	Set A (Pipeline)	Set B (Direct)
code 行數	62 / 68 / 36–156	210 / 294 / 51–895
spec (.md 字元)	5,376 / 6,789 / 3.6K–15K	11,746 / 15,593 / 5K–35K
own cells	0 / 532 / 0–4063	818 / 1,434 / 330–6275

[break-down](#)

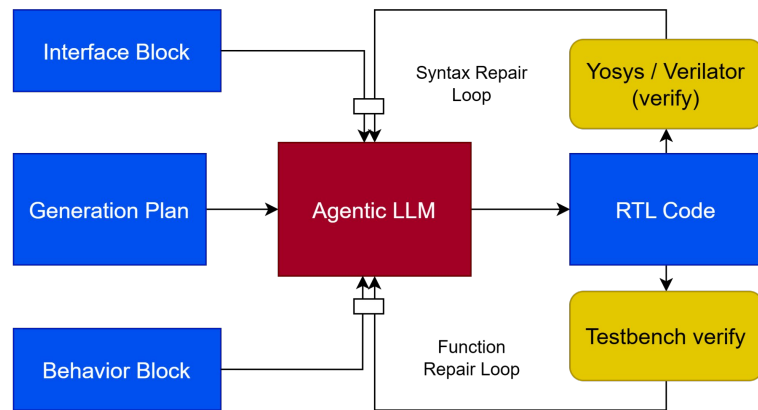
Router (Pipeline/Direct)

- So based on the characteristics, we build a router to categorize two different types of questions.
- the routing process can divide into several stages:
 - 1: For each spec, ask LLM to get some functional key attribute (delegates_to_submodules, is_memory, thin_control, has_fsm/cdc/algorithm ..etc.)
 - 2: Based on the attribute to decide to go pipeline or direct generation.
 - 2.5: If it is ambiguous to decide, generate the plan first.
 - 3: After plan generated, judge the intent of the module to decide whether generate it through pipeline or directly.



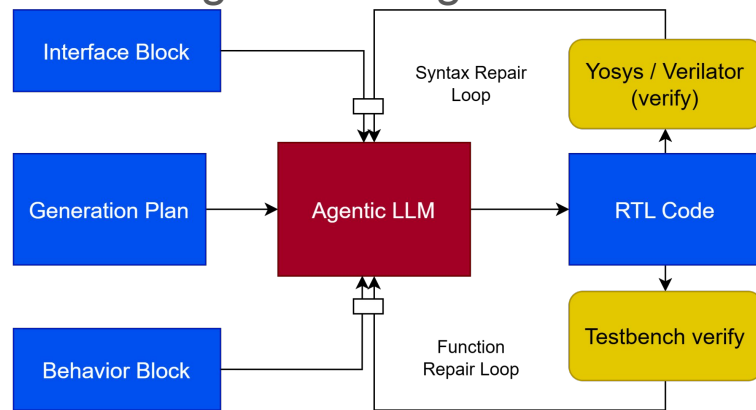
Functional Repair

- Action item: add functional repair into repair loop.
- After syntax check pass, we used to end the repair loop and evaluate the result.
- But now we start another functional repair loop to fix the functional error to raise the pass rate.
- We expect can have some performance gain coordinate with the spec slice & inject feature.



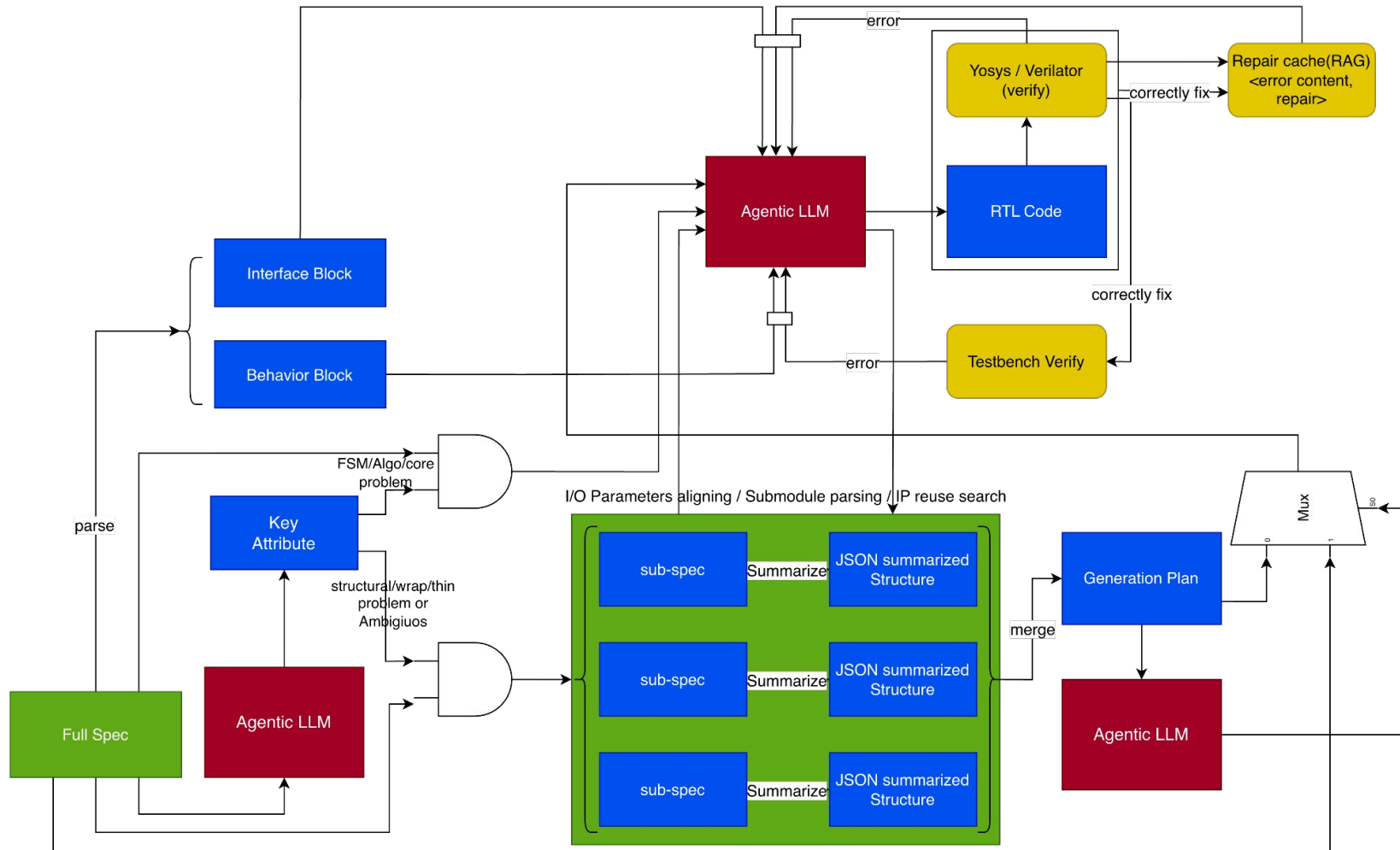
Spec Slicing

- [Why spec slice might be a must?](#)
- Spec slicing is a new feature that I added during the repair loop, we first slice the spe throught it specific format (markdown) into several topic, then we filter the topic into two category: interface & behavior.
- Interface information will be inject into repair loop when currently fixing the syntax error, aim to mimic the hallucination of the I/O signal naming.
- Behavior information will be inject into the functional repair loop to let LLM check where the logic is incorrectly implemented.



Tree Decomposition

- This is a feature that automatically divide the module into a hierarchical submodule tree through spec, and maintain a tree structure to control the generation/IP reuse process.
- After consideration, I put this stage before planning, which aim to help LLM arrange & understanding the detail for planning.
- 算了整合不進去，蛋掉



Evaluation

- V3 pipeline default
 - Repair Distribution
- Router
 - Accuracy
 - Performance

V3 Pipeline: Spec Slice & Functional Repair

- We evaluate the performance result of how spec slicing injection & functional repair helps generation.

	Syntax			Function		
	pass@1	pass@5	pass@10	pass@1	pass@5	pass@10
oss20B v2	66.92%	83.75%	87.73%	20.17%	27.40%	29.14%
oss20B v2 + Spec slicing	62.00%	81.84%	85.00%	20.67%	30.32%	33.33%
oss20B v2 + Spec slicing & Functional Repair = v3	60.33%	80.81%	85.00%	20.17%	30.01%	33.33%
oss20B v3 10 syntax repair 10 func repair	81.50%	90.65%	93.33%	27.50%	35.63%	38.33%
oss120B v2 pipeline	76.33%	89.49%	91.67%	24.50%	32.76%	35.00%
oss120B v3 10 syntax repair 10 func repair	89.83%	93.33%	93.33%	33.17%	39.50%	40.00%
qwen3-235B v3 10 syntax repair 10 func repair	63.67%	81.67%	81.67%	29.00%	33.33%	33.33%

Repair Distribution

- Can see that **syntax** gain keep decrease through each turn, a obvious decrease at 6 steps.
- Functional correctness gain is always low since lack of functional error information, which spec slicing may not help identify the error.
 - A possible guess is the testbench never output the clear error message to fix.
- So for cost efficiency, it is good to set the repair loop limits as **6/4** for syntax & function to avoid from endless resource wasting on few questions.

	0	1	2	3	4	5	6	7	8	9	10	Fail
syntax	231	120	48	28	20	20	8	7	7	9	5	97
function	255	5	3	3	2	3	0	3	0	0	0	326

Hybrid test on oss20B/120B

- This is the ability scale-up test on different version of pipeline with hybrid execution.
- Can see that v3 pipeline can help model to raise the accuracy compare to v2 pipeline.
-

	Samples	Syntax	Func pass	Solved	pass@1	pass@5	pass@10
20B v2	1200	45.80%	18.30%	30/60	18.30%	36.60%	43.00%
20B v3	1200	52.90%	21.70%	30/60	21.70%	40.90%	46.30%
120B v3	1199	73.10%	34.90%	35/60	34.90%	50.70%	55.50%

* v3 have a 10 times repair budget on both syntax/function (v2 only have 2 on syntax)

Router Result (Accuracy)

- Recall that pipeline can supply stable syntax pass (81%). Hybrid method's syntax pass drain since directly generate have a low pass rate(20-30%) and they compute as the average, but it can solve more various problem.
- With a router to distribute problem, can see that, we can maintain a not low syntax pass rate, with a new highest functional pass, and can see that, we only sample half of the hybrid method.
- Also need to notice when router route to directly generate, we still can run the syntax/function repair loop.

	Syntax			Function			Samples
	pass@1	pass@5	pass@10	pass@1	pass@5	pass@10	
oss20B v3 10 syntax repair 10 func repair	81.50%	90.65%	93.33%	27.50%	35.63%	38.33%	10
hybrid oss20B v3 10 syntax repair 10 func repair	52.92%	86.49%	90.12%	21.70%	40.90%	46.30%	10/10
Full-2T_router_oss20B_sync6_func4	72.67%	89.89%	91.67%	30.67%	50.30%	56.67%	10

Router Result (Performance)

- Since router can avoid redundant compute when it sufficient to been solved by directly generate, router version pipeline can gain more time/token reduction compare to other two pipeline.
- If accumulate the total generation time, we can gain a x3.09 speedup from using a router.

Run	Solved	Σ compute (h)	Pipeline tok	Direct tok (gen-only)	Total est	Pipeline LLM calls	tok / solved task
repair_spec (pipe-only, 10/10)	23/60	402	89.8M	—	89.8M	5,646	3.90M
hybrid (no-router)	30/60	423	89.8M	7.5M ⁺	97.3M ⁺	5,714	3.24M
Full-2T_router (routed, 6/4)	34/60	130	25.0M	3.3M ⁺	28.3M ⁺	1,469	0.83M

Action

- Model Capability Experiment
 - DeepSeek V3/R1 (large base model)
 - CodeV-R1, SiliconMind-V1 (verilog specific Model: pure model or with pipeline)
 - MAGE, VerilogCoder (other verilog coding methods comparison)
- 分析direct/pipeline 不同的 distribution, 分類器的正確性

名字

An Agentic Framework for RTL Code Generation with Design Planning, IP Reuse, and Closed-Loop Repair

結合設計規劃、IP 重用與閉迴路修復的代理式 Verilog 程式碼生成框架